# Large Language Model Assisted Human-AI Collaborative Development of Analytical Inverse Kinematics Solvers for Robot Manipulators

Preprint, compiled February 16, 2026

**Hai-Jun Su (su.298@osu.edu)**[1*]

[1]Department of Mechanical and Aerospace Engineering, The Ohio State University, Columbus, OH 43210, USA

## Abstract

This paper presents a large language model (LLM) assisted process for developing analytical closed-form inverse kinematics (IK) solvers for robot manipulators. The collaborative workflow enables human experts supervise AI agents through carefully designed prompts, while AI handles all coding, symbolic manipulation, and testing. This division of labor leverages each party's strengths: human intuition for problem decomposition and strategic guidance versus AI capability for systematic execution of labor-intensive tasks. The methodology comprises two key components: (1) a library of robust helper functions for common trigonometric equation systems, and (2) structured prompts that direct AI agents through derivation, code generation, and validation. Refined prompts significantly reduce iterative exchanges. We demonstrate the methodology on two major 6R robot architectures—spherical wrists and parallel joints—representing over 90% of industrial manipulators. The resulting solvers achieve 100% success rates across 842 industrial robots and 1000 random configurations, with working code produced in minutes rather than weeks. This work establishes a new paradigm where human expertise guides AI execution, significantly reducing development time while maintaining mathematical rigor.

## 1 Introduction

Large language models (LLMs) have rapidly advanced in capability, from the foundational Transformer architecture [1] and GPT-3's 175 billion parameters [2] to cutting-edge systems like GPT-5 [3] and autonomous coding agents such as OpenAI Codex that demonstrate exceptional reasoning, code generation, and multi-step problem solving through natural language prompts. These models exhibit emergent capabilities in mathematical reasoning and symbolic manipulation, enabling new possibilities for human-AI collaboration on complex technical tasks.

The application of LLMs to engineering and scientific problems has rapidly expanded, opening new paradigms for human-AI collaboration. Guo et al. [4] systematically benchmarked LLM capabilities across diverse engineering design tasks—including mechanism synthesis, structural analysis, and optimization. Romera-Paredes et al. [5] used LLMs to discover novel mathematical constructions, Trinh et al. [6] developed AlphaGeometry for solving International Mathematical Olympiad geometry problems. In robotics, Kim et al. [7] surveyed LLM integration with intelligent robots. In the broader context of machine learning for mechanism design, Deshpande and Purwar [8] demonstrated machine learning for planar four-bar linkage synthesis, and Purwar and Chakraborty [9] presented deep learning-driven design of robot mechanisms.

Despite this rapid progress, there currently exists no prior work on leveraging LLMs to solve robot kinematics problems—a fundamental analytical challenge in robotics that requires systematic equation manipulation, geometric reasoning, and robust numerical implementation. The commonly used methods include Jacobian-based iterative methods, polynomial homotopy continuation, algebraic elimination, and geometric subproblem decomposition. Jacobian-based numerical methods offer flexibility but only recover a single solution near an initial guess and can fail near singularities. The polynomial homotopy continuation method reliably tracks solution paths in the complex domain to find all real roots [10, 11], but is computationally expensive and less suited for real-time control. Geometric subproblem decomposition reduces IK into simpler subproblems, trading off generality, speed, and ease-of-maintenance in different ways. More recent toolchains such as IKFast [12] and IK-Geo [13] automate symbolic derivation or reduce IK into geometric subproblems. Ostermeier et al. [14] propose an automatic geometric decomposition that classifies manipulator kinematics online and reduces IK to pre-solved, numerically stable subproblems, enabling sub-millisecond derivation. While these algorithms can be fast and reliable, the development process still heavily relies on human expert knowledge and can be time-consuming.

Algebraic elimination and resultant-based techniques seek to systematically decouple unknowns and derive univariate characteristic polynomials. Extensive work on this topic are available in the literature. Representative results include the earlier work by Liao et al. ([15]), Lee and Liang [16], Raghavan–Roth framework [17]. They have successfully derived the 16th order polynomial for the most general 6R open chain or the equivalent 7R loop closed chain. To avoid the computing and solving the high-degree characteristic polynomial, Manocha and Zhu [18] converted the root-finding problem into a generalized eigenvalue problem to achieve an improved numerical stability. Since late 1980s and early 1990s, extensive research and numerous algorithms have been developed for the IK of 6R robot manipulators. Examples are Study's quadric formulation by Husty et al. [19], double quaternion formulation by Qiao et al. [20], sub-chain decomposition by Xiao et al. [21], and quaternions by Chen et al. [22]. However, the derivation process is often a heuristic, tedious, and error-prone task that requires significant expertise in robot kinematics, geometric insights, and substantial symbolic processing. While these solvers for general 6R robots have significant theoretical value, they are often far too complex and fail for real-world robots which typically

have special architectures leading to degenerated polynomial systems.

To automate the derivation process, Zhang and Hannaford developed IKBT which encodes engineers' symbolic reasoning as a behavior-tree of modular, knowledge-based solvers and automatically generates closed-form symbolic solutions together with a dependency graph that explains solution structure [23]. Learning-based and data-driven approaches have emerged as complementary directions: Malik et al. (2022) apply reinforcement-learning and policy-search techniques to learn inverse mappings for redundant manipulators, showing effective generalization on a 7-DOF Sawyer arm by treating IK as a learned control policy [24]. Ames et al. (IKFlow, 2022) cast IK as a generative sampling problem and use normalizing flows to rapidly produce diverse, high-quality IK candidates that cover multiple solution branches at millisecond runtimes [25]. These learning-based methods trade some analytical guarantees for speed and flexibility. However, they require substantial training data, may struggle with generalization to unseen robots or configurations, and lack the completeness guarantees of algebraic methods.

Despite many advances, inverse kinematics (IK) for 6R manipulators still faces practical tradeoffs of computational efficiency, numerical robustness, complexity and automation in derivation processes, and generalization of the final solver. The recently emerging large language models (LLMs) [1, 2] such as OpenAI's GPT series [26, 3] demonstrate markedly improved abilities in mathematical reasoning and coding. They are able to follow multi-step derivations, manipulate symbolic expressions, and produce clear chain-of-thought style explanations that support complex analytic workflows.

This paper addresses the shortfalls of current methods by presenting a novel LLM-assisted development process for analytical IK solvers. Unlike prior work that focuses on specific solver algorithms or robot-specific implementations, our primary contribution is the development methodology itself—a systematic, reproducible workflow that transforms the traditionally tedious and error-prone derivation process into a structured collaboration between human experts and AI coding agents. The process leverages LLMs to autonomously generate Computer Algebra System (CAS) scripts for symbolic derivation, implement numerical solvers in Python, and iteratively refine the code through validation feedback. Experts contribute high-level strategic guidance (e.g., decoupling strategies, elimination order) rather than performing detailed symbolic manipulations, fundamentally changing how their expertise is utilized. To demonstrate the generality and effectiveness of this development process, we apply it to two major 6R robot architectures: robots with spherical wrists and robots with parallel joints 2, 3, and 4. These architectures serve as case studies that collectively cover over 90% of industrial manipulators, but the methodology extends naturally to other robot configurations with identifiable geometric constraints.

## 2 RESEARCH METHODOLOGY

Developing analytical inverse kinematics solvers for 6R robots is a challenging task that involves extensive mathematical manipulations, numerous heuristic processes, and a tedious workflow, even when leveraging powerful computer algebra software. Many general solvers fail in singular, degenerated, or edge cases, requiring manual patching that is equally tedious and error-prone. The transition from research codes to a product-ready implementation is often fraught with difficulties.

Large language models (LLMs) are powerful tools for accelerating complex analytical derivations and code generation through natural language prompting. And the recently emerging advanced AI coding agents such as OpenAI's Codex can reason mathematically, call third-party tools, autonomously generate codes, iteratively debug and refine outputs based on validation feedback. This capability is particularly beneficial for developing analytical IK solvers, which involve intricate mathematical derivations and robust numerical implementations. In this paper, we present an LLM-assisted development process to address the aforementioned challenges for analytical inverse kinematics solvers.

### *2.1 Overview of the Development Process*

Figure 1 illustrates a flowchart of an iterative, expert-guided workflow for using LLM-based AI coding agents to develop IK solvers for 6R robots. The process begins with an expert providing the agent with the robot's Denavit-Hartenberg (DH) parameters, validation requirements, and domain-specific guidance on how to approach the IK problem. The AI agent first checks whether the robot satisfies known architectural constraints—such as having a spherical wrist or parallel joint axes—that allow analytic decoupling; if not, the process terminates or seek the human expert for additional guidance. Otherwise, the agent derives symbolic forward kinematics calling algebraic tools such as Mathematica or SymPy (python package for symbolic math), then applies the expert-defined strategy to identify decoupled equations and construct solvable subsystems. These subsystems are forwarded to specialized helper solvers which have been developed and rigorously tested with a separate process in prior, and the resulting joint solutions are back-substituted to recover remaining unknowns. Each candidate solution is validated through numerical forward-kinematics evaluation. If the solutions satisfy the validation criteria, the solver is accepted and returned. If not, the agent enters a debugging loop, refining the symbolic or numerical steps until either a valid solution is found or the maximum number of allowed iterations is reached, in which case the process returns to the expert for additional guidance.

**Prompt Design: Human Supervision Through Structured Communication** The cornerstone of human-AI collaboration in this work is prompt engineering—the systematic design of instructions that allow human experts to supervise and direct AI agents without performing tedious coding tasks themselves. Human experts encode their domain knowledge, strategic insights, and quality requirements into structured prompts, while AI agents execute all implementation details. We typically start from a casual description of the task, mathematical method, validation strategy, and deliverables. The human expert then refines this into a precise, execution-ready specification with the help of an LLM. This refined prompt is submitted to an autonomous coding agent (e.g., OpenAI Codex 5.1), which generates fully working, self-contained scripts within minutes.

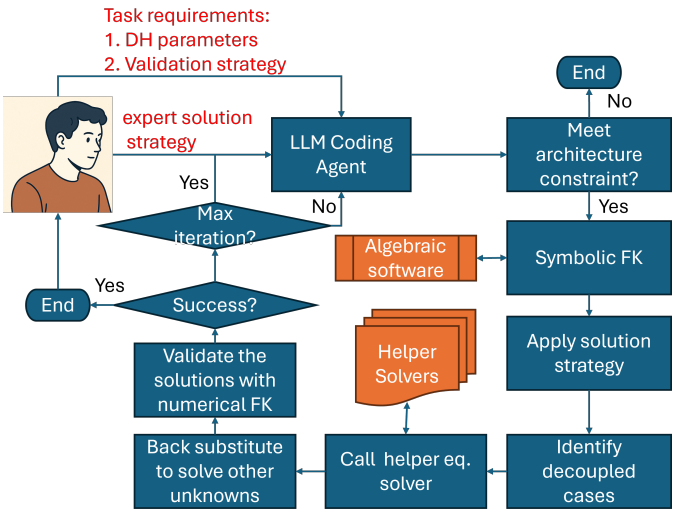


Figure 1: Flowchart describing the AI Agent-assisted development process for IK solvers.

The human expert reviews the output and provides follow-up prompts to address bugs, edge cases, or missing functionality—supervising rather than coding. For simple equation solvers, a well-structured prompt with four key blocks (context, task, requirements, deliverables) typically yields working code in one shot. For complex cases requiring symbolic derivation, the human expert iteratively guides the AI through follow-up prompts until all validation tests pass. This collaborative loop continues until the generated code handles all edge cases robustly. The key insight is that human experts contribute high-level strategic thinking and quality oversight, while AI agents perform all tedious symbolic manipulation, code writing, debugging, and testing.

The human-AI collaborative development process for IK solvers of 6R robots hinges on two major components:

1. **Helper Function Development (AI Executes, Human Supervises)**: Human experts define specifications for robust numerical solvers through structured prompts, while AI agents generate, test, and refine the implementations. These helper functions form a pre-validated library that higher-level IK solvers can call as black-box components. By having AI handle all coding and testing, human experts can focus on specifying requirements and validating correctness, dramatically accelerating development while maintaining quality.

2. **Expert Guidance Encoding (Human Thinks, AI Implements)**: Human experts provide strategic guidance on equation decoupling, variable elimination order, and geometric insights—the intellectual core that AI agents cannot reliably discover independently. This guidance is encoded in prompts that direct AI agents through symbolic derivation using computer algebra systems, code generation, and iterative refinement. The human expert supervises this process through follow-up prompts rather than manual coding.

In what follows, we detail each of these components and the specific prompting strategies employed to guide the AI agent-assisted development process.

### 2.2 *Helper Functions for Robust Equation Solving*

Inverse kinematics of serial robots frequently encounters several types of trigonometric equation systems that require specialized solution methods. The development process starts with developing numerically robust solvers for these equation types. Here we explicitly exclude numerical solvers such as Newton-Raphson due to their convergence issues and lack of the ability of computing the complete solution set. Also we exclude general polynomial system solvers such as Grobner basis and homotopy continuation methods due to their computational complexity and numerical instability. Here our focus is on reliable analytical methods including dialytic resultant elimination for solving these common equation types. See Figure 2 for the helper function development workflow.

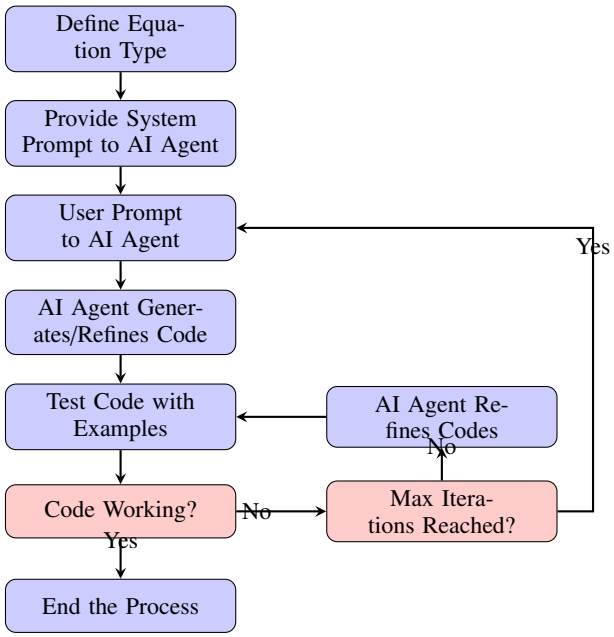


Figure 2: AI Agent-assisted helper function development flowchart.

Following the flowchart, a refined sample prompt for the single trigonometric equation solver (See Appendix A.1) is generated by LLM with one-shot prompting and is shown below:

**Context:** You are an expert mathematician and numerical analyst specializing in solving trigonometric equations that arise in robot kinematics.

**Task:** Implement a robust Python function that analytically solves the scalar equation `a*cos(x) + b*sin(x) + c = 0` using the Weierstrass substitution `t = tan(x/2)` to transform the equation into a quadratic in `t`, solve it, and map roots back to angles.

**Requirements:** 1. Handle every degenerate configuration, including cases with no trigonometric terms, purely sinusoidal/cosine equations, single-solution cases, and numerically tiny coefficients.

2. Return all valid real solutions (maximum of two) in a sorted list, with angles normalized to `[-pi, pi]`. Include a boolean flag indicating when the equation collapses to `0 = 0` (arbitrary angle).

3. Ensure all reported angles satisfy the original equation within a strict numerical tolerance and avoid false positives by re-evaluating each candidate in the original equation.

4. Add clear documentation explaining the mathematical derivation, all branch handling, and stability safeguards.

5. Include a lightweight validation routine that: (a) Generates at least 1000 random equations from known solution angles and asserts the solver recovers each root with high accuracy. (b) Always exercises explicit deterministic edge cases (degenerate, contradictory, single-solution, and endpoint cases) before the randomized trials. (c) Reports the number of randomized and deterministic tests executed.

6. Keep the implementation self-contained, rely only on the Python standard library, and aim for clear, production-ready code with concise comments explaining key reasoning steps.

**Deliverables:** Provide the final solver function `solve_trig_eq(a, b, c)`and the stress-test helper in a single Python module, following the requirements above.

The other three helper function types followed a similar development process. Table 1 summarizes the four trigonometric equation systems, their corresponding solver functions, maximum number of solutions, and the sections in Appendix A where their implementations are detailed. These functions implement numerically stable algorithms that handle degenerate cases, multiple solution branches, and edge conditions robustly. The detailed derivations, mathematical foundations, and comprehensive validation of these helper solvers are presented in our companion paper [27]. These prompts are tested in OpenAI's Codex 5.1 coding agent. The test results show that they can generate production-quality implementations in one shot with high probability. All the final refined prompts and their generated code are available for download in the GitHub repository listed in the section Data Availability.

### *2.3 Human Expert Supervision Through Prompting*

The human-AI collaborative process relies on human experts providing strategic supervision through prompts rather than performing tedious coding themselves. While AI agents excel at executing mathematical reasoning and generating code, they require human guidance for tractable, correct solver generation. Human experts contribute:

- principled task decomposition (e.g., isolating a preliminary development cycle to build robust, reusable trigonometric equation solvers before attempting full 6R IK);
- architectural architectural decoupling hints (e.g., recognizing spherical wrist intersection or parallel joint chains to separate position and orientation, or to reduce bilinear two-angle systems to quartics or planar 3R subproblems);
- expected solution cardinalities (e.g., up to 8 solutions for 6R robots with a spherical wrist or three parallel joint axes) used as immediate plausibility checks;
- explicit identification of eliminable variables and invariant geometric constraints; and
- targeted edge-case inventories (singular wrist, degenerate link lengths, colinear axes) to force the agent to exercise all numerical branches.

Without these expert priors encoded in prompts, AI agents often overfit to a single nominal case, miss solution branches, or silently drop roots during symbolic simplification. The prompts serve as the mechanism for human experts to inject their knowledge without performing manual coding.

The core of the human-AI collaboration for IK solvers lies in how human experts guide AI agents to strategically decouple equations and eliminate variables. The elimination strategy is problem-specific and requires expert knowledge of robot kinematics, geometric insights, and trigonometric identities—knowledge that human experts encode in prompts. The AI agent then executes the tedious work: deriving symbolic expressions, manipulating equations using computer algebra systems (CAS), and generating code. The human expert supervises this process, providing corrective prompts when the AI takes wrong paths.

Even the most powerful AI agents can get stuck due to the high complexity of symbolic manipulations and the large search space of possible elimination paths. This is precisely where human supervision becomes essential—not to do the work, but to guide the AI through prompts. Human experts provide hints on which variables to eliminate first, which geometric constraints to exploit, and which trigonometric identities to apply. These hints, encoded in prompts, narrow down the search space while AI agents execute all tedious manipulations. Example expert hints that humans encode in prompts include:

- **Variable elimination order**: Suggesting which joint angles to eliminate first based on their coupling structure in the equations. For instance, in a spherical wrist robot, eliminating the wrist joint angles (4, 5, 6) first can simplify the orientation equations significantly.
- **Simplification strategies**: Applying specific trigonometric identities (e.g., $\sin^2 + \cos^2 = 1$) to reduce expression complexity and reveal solvable patterns. Also if the elimination strategy leads to a dead end (symbolic expressions can explode during the derivation process), the expert can help the AI agent backtrack and explore alternative paths that may be more promising.
- **Recognition of solvable equation patterns**: Guide the AI agent to write temporary test scripts to analyze the derived symbolic expression and recognize equations or systems of equations that are solvable using known methods or the helper functions that have already been developed in prior. For example, identifying bilinear two-angle systems that can be solved using the dialytic elimination method.
- **Geometric insights**: Leveraging known geometric properties of the robot architecture to inform equation manipulation. For example, recognizing that certain link lengths or offsets lead to simplifications in the kinematic equations.
- **Inverse of transformation matrices**: Generic inverse of a 4x4 homogeneous transformation matrix in a symbolic form can be time consuming using SymPy. It can be replaced with an analytic SE(3) inverse to reduce complexity and improve numerical stability.

Table 1: Trigonometric Equation Systems, Their Solvers Detailed in Appendix

| Equation Type | Solver Function | # Sol. | Sec. |
|---|---|---|---|
| $a\cos\theta + b\sin\theta + c = 0$ | `solve_trig_eq()` | $\leq 2$ | A.1 |
| $[A_{2\times2}]\begin{pmatrix}\cos\theta\\ \sin\theta\end{pmatrix} = \mathbf{c}_{2\times1}$ | `solve_trig_sys_single()` | $\leq 1$ | A.2 |
| $[A_{2\times2}]\begin{pmatrix}\cos\theta_1\\ \sin\theta_1\end{pmatrix} + [B_{2\times2}]\begin{pmatrix}\cos\theta_2\\ \sin\theta_2\end{pmatrix} = \mathbf{c}_{2\times1}$ | `solve_trig_sys()` | $\leq 4$ | A.3 |
| $[K_{2\times9}]\mathbf{m} = \mathbf{0}$, where $\mathbf{m} = [1, c_1, s_1, c_2, s_2, c_1c_2, c_1s_2, s_1c_2, s_1s_2]^T$ | `solve_bilinear_sys()` | $\leq 8$ | A.4 |

- **Equation derivation**: Some equations can be derived more straightforwardly by considering the geometric relationships rather than purely algebraic manipulations. For example, equaling the angle between two adjacent z-axes the twist angle leads a target equation for solving.

Taking the IK solver for 6R robots with spherical wrists (See Section 4) as an example, we demonstrate how human experts supervise AI agents through prompting. Assuming the helper functions in Table 1 have been developed through prior human-AI collaboration, the human expert designs a prompt encoding strategic hints: how to decouple position and orientation subproblems, which variables to eliminate first, and which trigonometric identities to apply. Through iterative refinement—where the human reviews AI outputs and provides corrective prompts—a production-ready prompt emerges that generates fully functional solver code in a single shot. The human expert never writes solver code directly; instead, all implementation is performed by the AI agent under human supervision via prompts. The full prompt and generated code are available on GitHub. A condensed version is shown below:

**Context:** You are an analytical robotics expert guiding a coding agent to build a production-quality inverse kinematics solver for general 6R robots with spherical wrists at the last three joints. Helper solvers `solve_trig_sys` (two-angle) and `solve_trig_sys_single` (single-angle) already exist and must be reused.

**Task:** Write a Python script, plus any helper routines needed for the spatial 3R position subproblem and the spherical wrist orientation subproblem. The solver must enumerate every valid joint tuple, handle singularities and degeneracies, and ship with strong automated validation over random robots and poses.

**Requirements:**

1. Divide the IK problem into two subproblems: solving for the first three angles using the spatial 3R chain with the specified wrist center position, and solving for the last three joints using the spherical wrist orientation.

2. Preprocessing hint: Begin by "absorbing the effect of the base frame and the tool frame into the target pose" so the remaining pose describes the wrist center directly.

3. Spatial 3R hint: "Decouple $\theta_1$ from $\theta_3$ by moving the transformation of joint 1 to the left-hand side, then peel off joint 2 so only the wrist center kinematics of joints 3 and 4 remain." Use SymPy to derive the explicit coefficient expressions from the MDH parameters, analyze their structure to confirm the linear pattern, and hardcode the simplified forms in Python. Apply `solve_trig_sys` to solve $\theta_1$ from $\theta_3$ from two equations. Provide explicit guidance for symbolic manipulations: always apply $\sin^2 + \cos^2 = 1$ whenever a quadratic balloon starts forming, replace generic $4 \times 4$ inverses with the analytic SE(3) inverse.

4. Spherical wrist hint: "Cancel the effect of the first three joints to obtain the orientation seen from joint 4 and study how the wrist behaves when joint 5 tilts." From that reduced orientation, first solve $\theta_5$ by examining how the wrist's third axis aligns with the target. Next, treat $\theta_4$ and $\theta_6$ as single-angle problems by feeding the appropriate rotation matrix rows or columns into `solve_trig_sys_single`.

5. Symbolic-to-code workflow: Every time you introduce a new expression (pose simplification, wrist orientation entries, etc.), first derive it with CAS tooling (SymPy or Mathematica), inspect whether it fits the helper-solver pattern, then transcribe the final simplified form into Python so no symbolic evaluation remains at runtime.

6. Validation: Provide deterministic tests for edge scenarios (absorbing transformations with zero offsets, near-singular wrist twists, $\theta_5$ near zero, etc.) and a randomized stress campaign with at least 200 robot/pose samples. Report pass/fail counts plus any anomalous residuals.

**Deliverables:** A python script with preprocessing utilities, spatial 3R logic, wrist orientation solver, the public API, and inline comments narrating the elimination hints above.

## 3 PROBLEM STATEMENT

### 3.1 6R Robot Kinematics with Modified DH Parameters

We consider general 6R serial robots described by the Modified Denavit-Hartenberg (DHM) convention. The DHM parameters for each link are defined in Table 2.

Table 2: Modified Denavit-Hartenberg Parameters

| Link | $\alpha_{i-1}$ | $a_{i-1}$ | $d_i$ | $\theta_i$ |
|---|---|---|---|---|
| 1 | $\alpha_0$ | $a_0$ | $d_1$ | $\theta_1$ |
| 2 | $\alpha_1$ | $a_1$ | $d_2$ | $\theta_2$ |
| 3 | $\alpha_2$ | $a_2$ | $d_3$ | $\theta_3$ |
| 4 | $\alpha_3$ | $a_3$ | $d_4$ | $\theta_4$ |
| 5 | $\alpha_4$ | $a_4$ | $d_5$ | $\theta_5$ |
| 6 | $\alpha_5$ | $a_5$ | $d_6$ | $\theta_6$ |

The forward kinematics computes the end-effector pose ${}^0T_6$ as a homogeneous transformation matrix through successive link transformations:

$$ {}^0\mathbf{T}_6 = {}^0\mathbf{T}_1 \cdot {}^1\mathbf{T}_2 \cdot {}^2\mathbf{T}_3 \cdot {}^3\mathbf{T}_4 \cdot {}^4\mathbf{T}_5 \cdot {}^5\mathbf{T}_6 \tag{1} $$

where each ${}^i\mathbf{T}_{i+1}$ is given by:

$$ \begin{aligned} {}^i\mathbf{T}_{i+1} &= \mathbf{X}(\alpha_i, a_i) \cdot \mathbf{Z}(d_{i+1}, \theta_{i+1}) \\ &= \begin{bmatrix} c_{i+1} & -s_{i+1} & 0 & a_i \\ s_{i+1}c_{\alpha_i} & c_{i+1}c_{\alpha_i} & -s_{\alpha_i} & -s_{\alpha_i}d_{i+1} \\ s_{i+1}s_{\alpha_i} & c_{i+1}s_{\alpha_i} & c_{\alpha_i} & c_{\alpha_i}d_{i+1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned} $$

where $\mathbf{Z}(d, \theta)$ and $\mathbf{X}(\alpha, a)$ are the screw displacement matrices along z and x axes respectively and $c_{i+1} = \cos\theta_{i+1}$, $s_{i+1} = \sin\theta_{i+1}$, $c_{\alpha_i} = \cos\alpha_i$, and $s_{\alpha_i} = \sin\alpha_i$.

### 3.2 Inverse Kinematics Problem

Given a desired end-effector pose ${}^0\mathbf{T}_6^d = (\mathbf{R}^d, \mathbf{p}^d)$, the IK problem seeks joint angles $\theta = [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]^T$ such that the forward kinematics yields the desired pose.

First, transform the target pose ${}^0\mathbf{T}_6^d$ to a simplified coordinate frame $\mathbf{T}'_6$ by removing base and tool offsets:

$$ \mathbf{T}'_6 = \mathbf{Z}(-d_1, 0) \cdot \mathbf{X}(-\alpha_0, -a_0) \cdot {}^0\mathbf{T}_6^d \cdot \mathbf{Z}(-d_6, 0) \tag{2} $$

$$ = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3} $$

This transformation accounts for the base offset ($d_1$, $\alpha_0$, $a_0$) and tool offset ($d_6$), simplifying the subsequent kinematic analysis. From now on, this transformed target $\mathbf{T}'_6$ is then used in the IK solution process for both robot architectures in this paper.

For general 6R robots with special architectures such as spherical wrists or three parallel joints, this typically results in up to 8 real solutions due to the trigonometric nature of the equations. Our goal is to develop analytical closed-form solvers that:

1. Find all real solutions efficiently
2. Handle singularities and edge cases robustly
3. Work for general robot parameters within architectural constraints
4. Provide numerical stability and precision

## 4 INVERSE KINEMATICS OF GENERAL 6R ROBOTS WITH SPHERICAL WRIST

### 4.1 Robot Architecture and Constraints

Robots with spherical wrists (Fig. 3) are characterized by joints 4, 5, and 6 intersecting at a common point, resulting in the following DHM parameter constraints:

$$ a_4 = 0, \quad d_5 = 0, \quad a_5 = 0 $$

All other DHM parameters ($\alpha_i$, $a_i$, $d_i$, $\theta_i$ offsets) can be arbitrary, making this a general solver applicable to diverse robot morphologies beyond specific industrial models. See Figure 3 for an illustration. Unlike classical approaches that require the first 3R subchain to have special geometry, our solver treats the spatial 3R position subchain as fully general—only the spherical wrist constraints ($a_4 = d_5 = a_5 = 0$) are imposed.

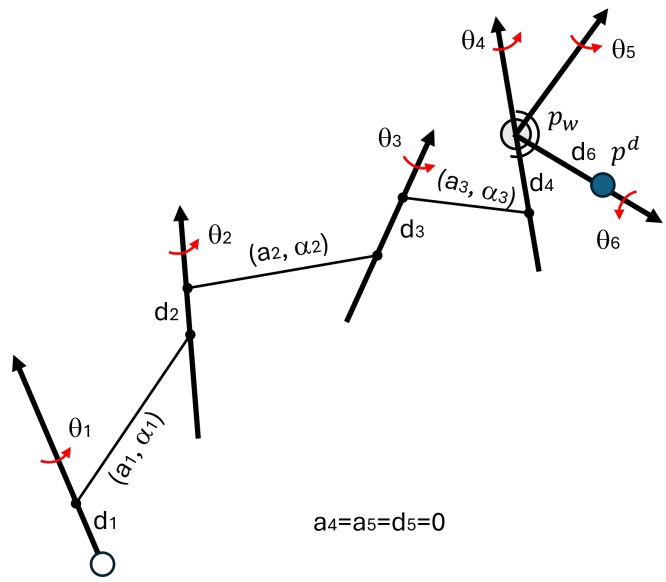


Figure 3: A general 6R robot with spherical wrist architecture.

### 4.2 IK Solution Strategy

The spherical wrist IK solver decomposes the problem into position and orientation subproblems, leveraging the kinematic decoupling enabled by the spherical wrist architecture. The algorithm follows these steps:

#### 4.2.1 Joints 1-3: Position Subproblem

Solve for the first three joint angles $\theta_1$, $\theta_2$, $\theta_3$ using the spatial 3R position IK solver with target position $\mathbf{p}_w$ which is the position part of $\mathbf{T}'_6$. This subproblem has been implemented in `Spatial3R_IK(`$\mathbf{p}_w$`)`.

To derive the coupled equations for $\theta_1$ and $\theta_3$, we begin by extracting position information from both sides of the decoupling equation. The z-component of the position vectors yields one constraint, while the magnitude of the position vectors provides a second constraint. Both constraints are then manipulated using standard trigonometric identities and algebraic simplification to eliminate other unknowns and isolate the terms involving the cosines and sines of $\theta_1$ and $\theta_3$. Through systematic collection of coefficients, both constraints are transformed into a unified linear form where all trigonometric terms are organized into matrix-vector products. This process naturally yields the coupled system of equations whcih are a linear form of $\sin\theta_1$, $\cos\theta_1$, $\sin\theta_3$ and $\cos\theta_3$ as the following:

$$ \mathbf{A} \begin{Bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{Bmatrix} + \mathbf{B} \begin{Bmatrix} \cos\theta_3 \\ \sin\theta_3 \end{Bmatrix} = \mathbf{C} \tag{4} $$

where $\mathbf{A}$, $\mathbf{B}$ are 2×2 coefficient matrices and $\mathbf{C}$ is a 2×1 constant vector, all independent of the joint angles. The detailed coefficient expressions are provided in [27].

The algorithm employs a systematic approach based on trigonometric elimination. First, a coupled system of two equations in $\theta_1$ and $\theta_3$ is derived from the kinematic constraints (Eq. 4). This system is solved using the helper solver `solve_trig_sys()`, which yields up to 4 real solution pairs depending on the robot geometry. For each $(\theta_1, \theta_3)$ pair, $\theta_2$ is recovered by substituting back into the position equations and solving the resulting single-angle trigonometric system using `solve_trig_sys_single()`. This decoupled approach ensures robust and complete enumeration of all feasible joint configurations for the spatial 3R position subproblem.

#### 4.2.2 *Joints 4-6: Orientation Subproblem*

The spherical wrist assumes intersecting joint axes with zero link lengths and offsets. However the twist angles $\alpha_4$ and $\alpha_5$ may be non-zero and arbitrary (may be different to ±90°). First , compute the transformation from frame 0 to frame 4 using the solved first three joint angles: ${}^4\mathbf{T}_6^d = {}^0\mathbf{T}_4^{-1} \cdot \mathbf{T}'_6$, where $\mathbf{T}'_6$ is the modified target end-effector pose. Let the desired rotation matrix ${}^4\mathbf{R}_6^d$ be the rotation part of ${}^4\mathbf{T}_6^d$.

The spherical wrist forward kinematics gives the rotation matrix from frame 4 to frame 6:

$$ {}^4\mathbf{R}_6^d = \mathbf{R}_z(\theta_4) \cdot \mathbf{R}_x(\alpha_4) \cdot \mathbf{R}_z(\theta_5) \cdot \mathbf{R}_x(\alpha_5) \cdot \mathbf{R}_z(\theta_6) $$

The orientation subproblem then solves the above equation for $\theta_4, \theta_5, \theta_6$ given the desired rotation matrix ${}^4\mathbf{R}_6^d$.

The solution for the spherical wrist configuration begins by solving for $\theta_5$ using the [3,3] element of the desired rotation matrix, $r_{33}$, which relates to $\alpha_4$, $\alpha_5$, and $\theta_5$. This yields two solutions for $\theta_5$, excluding cases where $\sin(\alpha_4)\sin(\alpha_5) = 0$. For each $\theta_5$, we solve for $\theta_4$ using the [3,1] and [3,2] elements of the rotation matrix, forming a trigonometric system expressed as a matrix equation. The helper function `solve_trig_sys_single` provides a unique solution for $\theta_4$ for each $\theta_5$. Finally, $\theta_6$ is determined using the [1,3] and [2,3] elements of the rotation matrix, again solved with `solve_trig_sys_single`. If $\sin(\theta_5) = 0$, we have infinite solutions, allowing us to set either $\theta_4$ or $\theta_6$ arbitrarily while solving for the other. This approach yields up to 2 solutions for the spherical wrist subproblem, corresponding to the two branches of $\theta_5$.

The overall IK algorithm combines the position and orientation subproblems to yield the complete joint angle solutions for the 6R robot with spherical wrist. The pseudocode is provided in Algorithm 1. The solver of this subproblem is implemented in `SphericalWrist_IK`$({}^4\mathbf{R}_6^d)$.

**Algorithm 1** Inverse Kinematics for 6R Robots with Spherical Wrist

**Input:** Target pose $\mathbf{T}_6^d = (R^d, p^d)$, DH parameters
**Output:** Joint angle solutions $[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$
{Step 1: Compute the simplified transformation matrix $\mathbf{T}'_6$}

{Step 2: Extract wrist center position $p_w$ from $\mathbf{T}'_6$}
$p_w \leftarrow$ ExtractPosition($\mathbf{T}'_6$)
{Step 3: Solve position subproblem (joints 1-3)}
$([\theta_1, \theta_2, \theta_3]) \leftarrow$ Spatial3R_IK($p_w$)
**for** each solution $(\theta_1, \theta_2, \theta_3)$ **do**
  {Step 4: Compute transformation to link 3}
  $\mathbf{T}_3 \leftarrow$ ForwardKinematics$(\theta_1, \theta_2, \theta_3)$
  {Step 5: Extract required wrist orientation}
  ${}^4\mathbf{T}_6^d \leftarrow {}^0\mathbf{T}_3^{-1} \cdot \mathbf{T}'_6$
  ${}^4\mathbf{R}_6^d \leftarrow {}^4\mathbf{T}_6^d[1:3, 1:3]$
  {Step 6: Solve orientation subproblem (joints 4-6)}
  $[\theta_4, \theta_5, \theta_6] \leftarrow$ SphericalWrist_IK$({}^4\mathbf{R}_6^d)$
  **for** each orientation solution $(\theta_4, \theta_5, \theta_6)$ **do**
    {Step 7: Combine and validate solution}
    solution $\leftarrow [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$
    $\mathbf{T}_{check} \leftarrow$ ForwardKinematics(solution)
    **if** $\|\mathbf{T}_{check} - \mathbf{T}_6^d\| < \epsilon$ **then**
      Add solution to valid solutions list
    **end if**
  **end for**
**end for**
**return** List of valid joint angle solutions

## 5 INVERSE KINEMATICS OF GENERAL 6R ROBOTS WITH PARALLEL JOINT STRUCTURES

### 5.1 *Robot Architecture and Constraints*

Collaborative robots frequently employ parallel joint structures where joints 2, 3, and 4 have parallel axes. This architecture is characterized by:

$$ \alpha_2 = 0^\circ, \quad \alpha_3 = 0^\circ $$

All other DHM parameters remain arbitrary, enabling the solver to handle diverse robot designs within this architectural family. Unlike existing solvers that often assume additional special geometry beyond the parallel constraint, our solver requires only the minimal condition $\alpha_2 = \alpha_3 = 0$; all other DH parameters remain fully arbitrary.

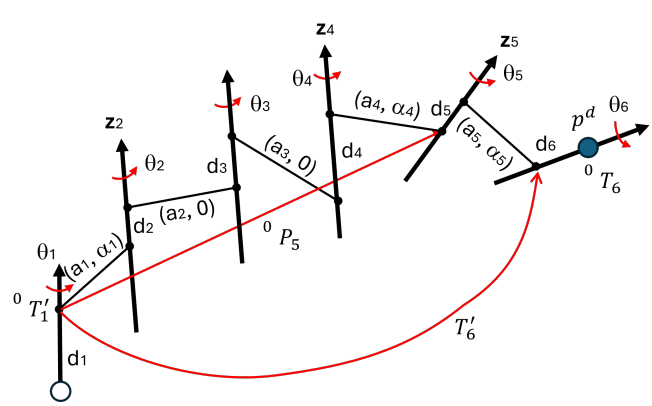


Figure 4: Illustration of a 6R robot with parallel joint architecture.

### 5.2 IK Solution Strategy

The parallel joint IK solver employs a decoupled approach with three main stages:

#### 5.2.1 Bilinear System for Joints 1 and 6

Solve the coupled $\theta_1$, $\theta_6$ variables using bilinear equations derived from the kinematic constraints. The solution process distinguishes between coupled and decoupled cases based on the link parameters $a_5$ and $d_5$.

**Derivation of the Bilinear Equations** The bilinear equations arise from geometric constraints in the parallel joint architecture. Break the FK chain into two parts: the first four joints (1-4) and the last two joints (5-6). The constraints are derived as follows:

$$ {}^0\mathbf{T}_4 = {}^0\mathbf{T}'_1 \cdot {}^1\mathbf{T}_2 \cdot {}^2\mathbf{T}_3 \cdot {}^3\mathbf{T}_4, \tag{5} $$

$$ {}^0\mathbf{T}_5 = \mathbf{T}'_6 \cdot {}^5\mathbf{T}_6{}^{-1}, \tag{6} $$

$$ {}^0\mathbf{T}_2 = {}^0\mathbf{T}'_1 \cdot {}^1\mathbf{T}_2, \tag{7} $$

where ${}^0\mathbf{T}'_1$ the simplified transformation of the first joint by setting $\alpha_0 = a_0 = d_1 = 0$ and $\mathbf{T}'_6$ is defined in Eq. (3).

Let's define the joint axes used in the constraint equations:

$$ \mathbf{z}_4 = {}^0\mathbf{T}_4[1:3,3], \tag{8} $$

$$ \mathbf{z}_5 = {}^0\mathbf{T}_5[1:3,3], \tag{9} $$

$$ \mathbf{z}_2 = {}^0\mathbf{T}_2[1:3,3] \tag{10} $$

Consider the joint axes: $z_4$ and $z_5$ are the axes of joints 4 and 5, respectively, with relative angle $\alpha_4$. The axis $z_2$ is the axis of joint 2.

**Equation 1: Orientation constraint** is defined by the angle between joint axes 4 and 5 equaling $\alpha_4$:

$$ \mathbf{z}_4 \cdot \mathbf{z}_5 - \cos(\alpha_4) = 0 \tag{11} $$

**Equation 2** is derived from the position constraint that the projection of the difference between the two expressions for frame 5's position onto joint axis 2 must be zero. First, compute $0\mathbf{T}_5$ from both sides of the kinematic chain:

$$ \mathbf{T}'_6 \cdot {}^5\mathbf{T}_6{}^{-1} = {}^0\mathbf{T}'_1 \cdot {}^1\mathbf{T}_2 \cdot {}^2\mathbf{T}_3 \cdot {}^3\mathbf{T}_4 \cdot {}^4\mathbf{T}_5 \tag{12} $$

Second, we define the two points $\mathbf{P}^{\mathrm{L}}_{05}$ and $\mathbf{P}^{\mathrm{R}}_{05}$ as the position of frame 5 computed from the left and right sides, respectively.

Since point $\mathbf{P}_{05}$ is moving on a plane formed by the three parallel axes of joint 2, 3 and 4, that is perpendicular to $\mathbf{z}_2$, we project the point difference onto $\mathbf{z}_2$ to form the second constraint:

$$ (\mathbf{P}^{\mathrm{R}}_{05} - \mathbf{P}^{\mathrm{L}}_{05}) \cdot \mathbf{z}_2 = 0 \tag{13} $$

Note that $\mathbf{P}^{\mathrm{R}}_{05} \cdot \mathbf{z}_2$ is a constant computed as:

$$ \mathbf{P}^{\mathrm{R}}_{05} \cdot \mathbf{z}_2 = d_2 + d_3 + d_4 + d_5 \cos(\alpha_4) $$

And $\mathbf{P}^{\mathrm{L}}_{05} \cdot \mathbf{z}_2$ is independent of $\theta_5$. Both constraints are in the bilinear form of cosine and sine of joint angles $\theta_1$ and $\theta_6$ as follows:

$$ \begin{bmatrix} a_{10} & a_{11c} & a_{11s} & a_{12c} & a_{12s} & a_{1cc} & a_{1cs} & a_{1sc} & a_{1ss} \\ a_{20} & a_{21c} & a_{21s} & a_{22c} & a_{22s} & a_{2cc} & a_{2cs} & a_{2sc} & a_{2ss} \end{bmatrix} \mathbf{m} = \mathbf{0} \tag{14} $$

where the coefficients $a_{kij}$ are defined in Table 8 in Appendix B. The monomial vector $\mathbf{m}$ is defined as:

$$ \mathbf{m} = (1, c_1, s_1, c_6, s_6, c_1c_6, c_1s_6, s_1c_6, s_1s_6)^T $$

where $c_1 = \cos\theta_1$, $s_1 = \sin\theta_1$, $c_6 = \cos\theta_6$, $s_6 = \sin\theta_6$.

**Coupled Case** ($a_5 \neq 0$) When link parameters $a_5$ is non-zero, the equations remain the coupled form. The coefficients are computed symbolically using Mathematica and hardcoded in the implementation of the bilinear solver `solve_bilinear_sys()`, which employs Sylvester resultants and generalized eigenvalue methods to find up to 8 solution pairs $(\theta_1, \theta_6)$. Only up to 4 real solutions are expected for typical robot geometries. The other solutions are extraneous and discarded. See Section A.4 for details on the bilinear solver implementation.

**Decoupled Case** ($a_5 = 0$) When both $a_5$ and $d_5$ are zero, the equations decouple, allowing sequential solution. Eq. (13) simplifies to a single trigonometric equation in $\theta_1$:

$$ a\cos\theta_1 + b\sin\theta_1 + c = 0 $$

which is solved using `solve_trig_eq()` to obtain up to 2 solutions for $\theta_1$. For each $\theta_1$ solution, Eq. (11) becomes a trigonometric equation in $\theta_6$ which has a similar form and can be also solved with `solve_trig_eq()`. This yields up to 2 solutions for $\theta_6$ per $\theta_1$, for a total of up to 4 solution pairs.

#### 5.2.2 Joint 5 Solution

With $\theta_1$ and $\theta_6$ known, solve for $\theta_5$ using the matrix equation Eq. (12). Basically, we pick two equations with $\theta_5$ dependence from the position part of the matrix equation and rearrange them into a single-angle trigonometric system and then solve using `solve_trig_sys_single()`. This yields a unique $\theta_5$ for each $(\theta_1, \theta_6)$ pair.

#### 5.2.3 Planar 3R IK for Joints 2, 3, 4

With $\theta_1$, $\theta_5$, and $\theta_6$ known, solve for the remaining joints $\theta_2$, $\theta_3$, $\theta_4$ using planar kinematics. The parallel joint constraint ($\alpha_2 = \alpha_3 = 0$) simplifies this to a 2D problem in the plane perpendicular to the joint axes.

Rearrange the FK chain Eq. (12) to isolate the planar 3R subproblem:

$$ {}^0\mathbf{T}'^{-1}_1 \cdot \mathbf{T}'_6 \cdot {}^5\mathbf{T}_6{}^{-1} \cdot {}^4\mathbf{T}^{-1}_5 = {}^1\mathbf{T}_2 \cdot {}^2\mathbf{T}_3 \cdot {}^3\mathbf{T}_4 \tag{15} $$

The left hand side is ${}^2\mathbf{T}^L_4$ computed using the computed joint angles $\theta_1, \theta_5, \theta_6$ and the target pose $\mathbf{T}'_6$ from the previous stages. The right hand side represents the planar 3R forward kinematics with unknowns $\theta_2$, $\theta_3$, $\theta_4$, which has the following form:

$$ {}^2\mathbf{T}^R_4 = \begin{bmatrix} c_{234} & -s_{234} & 0 & a_2c_2 + a_3c_{23} \\ s_{234} & c_{234} & 0 & a_2s_2 + a_3s_{23} \\ 0 & 0 & 1 & d_3 + d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

where $c_{234} = \cos(\theta_2 + \theta_3 + \theta_4)$, $s_{234} = \sin(\theta_2 + \theta_3 + \theta_4)$, $c_{23} = \cos(\theta_2 + \theta_3)$, and $s_{23} = \sin(\theta_2 + \theta_3)$.

The goal is to solve $\theta_2, \theta_3, \theta_4$ that satisfy ${}^2\mathbf{T}^L_4 = {}^2\mathbf{T}^R_4$. The solution proceeds as follows:

1. Extract the planar position $(p_x, p_y)$ from $\mathbf{T}_5^L[1:2,4]$
2. Solve for $\theta_3$ using the law of cosines:
$$p_x^2 + p_y^2 = a_2^2 + a_3^2 + 2a_2a_3c_3 \tag{16}$$
yielding two solutions (elbow-up/elbow-down configurations)
3. For each $\theta_3$, solve for $\theta_2$ from the position equations:
$$p_x = a_2 \cos\theta_2 + a_3 \cos(\theta_2 + \theta_3) \tag{17}$$
$$p_y = a_2 \sin\theta_2 + a_3 \sin(\theta_2 + \theta_3) \tag{18}$$
using the helper `solve_trig_sys_single()` to obtain a unique $\theta_2$ for each $\theta_3$.
4. Extract the cumulative angle $\theta_{234} = \texttt{atan2}(\mathbf{T}_5^L[2,1], \mathbf{T}_5^L[1,1])$
5. Compute $\theta_4 = \theta_{234} - \theta_2 - \theta_3$

This approach yields up to 2 solutions for $(\theta_2, \theta_3, \theta_4)$ per $(\theta_1, \theta_5, \theta_6)$ combination.

The overall IK algorithm for 6R robots with parallel joints combines the bilinear system solution for joints 1 and 6, orientation constraint solving for joint 5, and planar kinematics for joints 2, 3, 4. This architecture-specific approach leverages the parallel joint constraint to achieve robust solutions with up to 8 total configurations. The complete pseudocode is provided in Algorithm 2.

**Algorithm 2** Inverse Kinematics for 6R Robots with Parallel Joints (Joints 2, 3, 4)

**Input:** Target pose $\mathbf{T}_6^d = (R^d, p^d)$, DH parameters
**Output:** Joint angle solutions $[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$
{Step 1: Validate parallel joint constraints}
**if** $\alpha_2 \neq 0$ **or** $\alpha_3 \neq 0$ **then**
  **return** Error: Joints 2, 3, 4 must be parallel
**end if**
{Step 2: Pre-process pose for non-zero base/tool parameters}
**if** $\alpha_0 \neq 0$ **or** $a_0 \neq 0$ **or** $d_1 \neq 0$ **or** $d_6 \neq 0$ **then**
  $\mathbf{T}_6^d \leftarrow \mathbf{T}_{base}^{-1} \cdot \mathbf{T}_6^d \cdot \mathbf{T}_{tool}^{-1}$
**end if**
{Step 3: Solve bilinear system for joints 1 and 6}
$([\theta_1, \theta_6]) \leftarrow \text{SolveBilinearSystem}(\mathbf{T}_6^d)$
**for** each solution $(\theta_1, \theta_6)$ **do**
  {Step 4: Solve for joint 5 from orientation constraint}
  $\theta_5 \leftarrow \text{SolveJoint5}(\mathbf{T}_6^d, \theta_1, \theta_6)$
  **if** $\theta_5$ is valid **then**
    {Step 5: Solve planar 3R IK for joints 2, 3, 4}
    $([\theta_2, \theta_3, \theta_4]) \leftarrow \text{Planar3R_IK}(\mathbf{T}_6^d, \theta_1, \theta_6, \theta_5)$
    **for** each planar solution $(\theta_2, \theta_3, \theta_4)$ **do**
      {Step 6: Combine and validate solution}
      solution $\leftarrow [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$
      $\mathbf{T}_{check} \leftarrow \text{ForwardKinematics}(\text{solution})$
      **if** $\|T\mathbf{T}_{check} - \mathbf{T}_6^d\| < \epsilon$ **then**
        Add solution to valid solutions list
      **end if**
    **end for**
  **end if**
**end for**
**return** List of valid joint angle solutions

## 6 BENCHMARK TESTS

Two kinds of tests were conducted to validate the solvers: (1) real robot validation using industrial robot models from the RobotKinematicsCatalogue database (https://github.com/SaltworkerMLU/RobotKinematicsCatalogue). These robots typically have special geometric parameters like 90 degrees for twist angles, 0 link length etc., and (2) stress testing with selected degenerate/edge cases of DHM parameters as well as randomly generated DHM parameters. The complete prompts, test scripts, and detailed benchmark results for the spherical wrist solver are available at https://github.com/haijunsu-osu/llm_ik_6r_wrist, and those for the parallel axes solver are available at https://github.com/haijunsu-osu/ik_6r_parallel.

All the solvers and test codes were implemented in Python. All tests were performed on a standard desktop computer with an Intel i7-10700KF CPU @ 3.80GHz and 32GB RAM. Each IK solve was timed using high-resolution timers, and the average solve time was computed over multiple runs to ensure accuracy.

### 6.1 *General 6R Spherical Wrist Testing*

Table 3 summarizes the validation results for the spherical wrist solver. We tested it on 723 industrial robots from the RobotKinematicsCatalogue database for real robot validation each with 10 random poses per robot/configuration. We also tested 1000 randomly generated DHM parameter sets each with 20 random poses for stress testing. The solver achieved a 100% success rate in both test types, with average solve times well below 2 ms and maximum position errors on the order of $10^{-9}$ or lower, demonstrating high accuracy and efficiency.

Table 3: 6R Spherical Wrist Solver Validation Results

| Test Type | Robots | Tests/ Robot | Succ. Rate | Avg. Time | Max Error |
|---|---|---|---|---|---|
| Real Robots | 723 | 10 | 100% | 1.22 ms | 2.19e-09 |
| Random Robots | 1000 | 20 | 100% | 1.17 ms | 2.68e-11 |

### 6.2 *General 6R Parallel Joint Testing*

Table 4 summarizes the validation results for the parallel joint solver. We tested it on 119 collaborative robots from the RobotKinematicsCatalogue database for real robot validation, each with 10 random poses per robot. We also tested 100 randomly generated DHM parameter sets (maintaining parallel joint constraints) each with 10 random poses for stress testing. The solver achieved a 100% success rate in both test types, with average solve times below 2 ms. The stress tests showed robust edge case handling and slightly faster solve times of 1.12 ms. Maximum position errors remained on the order of $10^{-9}$ or lower across all tests, confirming high numerical accuracy.

### 6.3 *Discussion of Computational Accuracy and Efficiency*

The benchmark results demonstrate both high accuracy and computational efficiency of the developed solvers. The maximum position errors across all tests are on the order of $10^{-9}$ to $10^{-11}$, which are near the limits of double-precision floating-

Table 4: 6R Parallel Joint Solver Validation Results

| Test Type | Robots | Tests/ Robot | Succ. Rate | Avg. Time | Max Error |
|---|---|---|---|---|---|
| Real Robots | 119 | 10 | 100% | 1.79 ms | 3.45e-09 |
| Random Robots | 100 | 10 | 100% | 1.12 ms | 1.82e-09 |

point arithmetic. This level of accuracy confirms the analytical nature and numerical stability of the solvers—no iterative refinement or convergence criteria are needed.

In terms of computational efficiency, the average solve times of 1.12–1.79 ms in Python are already sufficient for many real-time robot control applications, where control loop periods typically range from 1–10 ms. The Python implementation prioritizes code clarity and correctness; substantial speedup (typically 10–100×) can be achieved by translating the solvers to a compiled language such as C++, which would bring solve times to the sub-100 $\mu$s range.

### 6.4 *Comparison with Alternative Approaches*

Compared with alternative IK solution approaches, our method offers distinct advantages across several key dimensions. Jacobian-based numerical methods are iterative, find only a single solution per invocation, and can fail near singularities; our analytical solvers find all solutions (up to 8) with near machine-precision accuracy ($\sim 10^{-9}$) and robust singularity handling. Polynomial homotopy continuation methods reliably find all roots but are computationally expensive (~10–100 s), making them less suitable for real-time control; our solvers achieve ~1–2 ms solve times in Python with no training data required. IKFast and IK-Geo automate parts of the derivation process but still require significant manual setup and may not handle all degenerate cases robustly; our solvers have been stress-tested on thousands of configurations including degenerate cases. Learning-based approaches (e.g., IKFlow) require large training datasets, may not generalize to unseen robots, and achieve lower accuracy ($\sim 10^{-3}$) without completeness guarantees.

The key advantages of our approach are: (1) dramatically faster development cycle enabled by the human-AI collaborative methodology; (2) full generality within each architectural family (arbitrary DH parameters beyond the minimal architectural constraints); (3) analytical completeness with all solutions found; and (4) near machine-precision accuracy without iterative convergence. The main limitation is that the current Python implementation, while sufficiently fast for many applications, can be further optimized through C++ translation for the most demanding real-time control scenarios.

## 7 CONCLUSIONS

This paper presents a LLM based human-AI collaborative framework for developing analytical inverse kinematics solvers for 6R robot manipulators. The key innovation is a division of labor where human experts provide strategic supervision through structured prompts while AI agents execute all coding, symbolic manipulation, and testing tasks.

We developed analytical inverse kinematics solvers for two major 6R robot architectures—spherical wrists and parallel joints—covering over 90% of industrial manipulators. The methodology leverages human experts encoding strategic guidance through structured prompts while AI agents execute all symbolic derivation, code generation, and validation. The resulting solvers achieved 100% success rates across 842 industrial robots and 1000 randomly generated configurations, with average solve times of 1.1–1.8 ms and numerical accuracy near machine precision ($\sim 10^{-9}$). All code, prompts, and test scripts are publicly available on GitHub. Future improvements include optimizing performance through C++ implementation and extending the methodology to additional robot architectures.

The main contributions are: (1) a reproducible human-AI collaborative development process that reduces solver creation from weeks to minutes while maintaining mathematical rigor, and (2) general-purpose analytical IK solvers for two major robot families with minimal architectural constraints. This work establishes a new paradigm for engineering software development, demonstrating that AI-assisted development can dramatically accelerate complex mathematical problems. By automating tedious implementation while preserving human expertise in strategic guidance, the methodology reduces development time and establishes a blueprint for human-AI collaboration on analytical problems across robotics and engineering domains.

## DECLARATION OF COMPETING INTERESTS

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## DATA AVAILABILITY

The source code for all inverse kinematics solvers and helper functions is publicly available on GitHub at `https://github.com/haijunsu-osu/kinematics_equation_solvers`. The complete prompts, AI-generated Python scripts, and benchmark test results for the spherical wrist solver are available at `https://github.com/haijunsu-osu/llm_ik_6r_wrist`, and those for the parallel axes solver are available at `https://github.com/haijunsu-osu/ik_6r_parallel`. The robot kinematic data used for validation is available from the RobotKinematicsCatalogue repository (`https://github.com/SaltworkerMLU/RobotKinematicsCatalogue`).

## FUNDING

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE MANUSCRIPT PREPARATION PROCESS

During the preparation of this work, the author used OpenAI GPT and Claude (Anthropic) coding assistant in order to de-

velop the inverse kinematics solver code and helper functions, as well as to assist with symbolic derivations for the trigonometric equation solvers. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

## REFERENCES


[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, M. Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, G. Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[3] OpenAI. Openai GPT-5 system card. *arXiv preprint arXiv:2601.03267*, 2025. URL https://arxiv.org/abs/2601.03267.

[4] Xingang Guo, Yaxin Li, Xiangyi Kong, Yilan Jiang, Xiayu Zhao, Zhihua Gong, Yufan Zhang, Daixuan Li, Tianle Sang, Beixiao Zhu, et al. Toward engineering AGI: Benchmarking the engineering design capabilities of LLMs, 2025. URL https://arxiv.org/abs/2509.16204.

[5] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[6] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024. doi: 10.1038/s41586-023-06747-5.

[7] Yoonsoo Kim, Dongkeun Kim, Jongrae Choi, Jaeheung Park, Namhoon Oh, and Dongwoon Park. A survey on integration of large language models with intelligent robots. *Intelligent Service Robotics*, 17:1103–1125, 2024. doi: 10.1007/s11370-024-00550-5.

[8] Shrinath Deshpande and Anurag Purwar. A machine learning approach to kinematic synthesis of defect-free planar four-bar linkages. *ASME Journal of Computing and Information Science in Engineering*, 19(2):021004, 2019. doi: 10.1115/1.4042325.

[9] Anurag Purwar and Nilanjan Chakraborty. Deep learning-driven design of robot mechanisms. *ASME Journal of Computing and Information Science in Engineering*, 23 (6):060811, 2023. doi: 10.1115/1.4062626.

[10] L.-W. Tsai and A. P. Morgan. Solving the kinematics of the most general six- and five-degree-of-freedom manipulators by continuation methods. *ASME Journal of Mechanisms, Transmissions, and Automation in Design*, 107(2): 189–200, June 1985.

[11] Charles W. Wampler and Alexander P. Morgan. Solving the kinematics of manipulators using linear homotopies. *Journal of Mechanical Design*, 108(1):59–68, 1986.

[12] Rosen Diankov. *Automated Construction of Robotic Manipulation Programs*. PhD thesis, Carnegie Mellon University, 2010.

[13] Alexander J. Elias and John T. Wen. Ik-geo: Unified robot inverse kinematics using subproblem decomposition. *Mechanism and Machine Theory*, 209:105971, 2025. ISSN 0094-114X. doi: https://doi.org/10.1016/j.mechmachtheory.2025.105971. URL https://www.sciencedirect.com/science/article/pii/S0094114X25000606.

[14] Daniel Ostermeier, Jonathan Külz, and Matthias Althoff. Automatic geometric decomposition for analytical inverse kinematics. *IEEE Robotics and Automation Letters*, 10(10):9964–9971, 2025. doi: 10.1109/LRA.2025.3597897.

[15] Q. Z. Liao, C. G. Liang, and Q. X. Zhang. A novel approach to the displacement analysis of general spatial 7r mechanism. *Chinese Journal of Mechanical Engineering*, 22(3):1–9, September 1986.

[16] Hong-You Lee and Chong-Gao Liang. Displacement analysis of the general spatial 7-link 7r mechanism. *Mechanism and Machine Theory*, 23(3):219–226, 1988.

[17] M. Raghavan and B. Roth. Kinematic analysis methods for serial manipulators (raghavan and roth method). *Various (see Raghavan and Roth)*, 1990.

[18] Dinesh Manocha and Yunshan Zhu. A fast algorithm and system for the inverse kinematics of general serial manipulators. In *Proceedings / IEEE*, 1994.

[19] Manfred L. Husty, Martin Pfurner, and Hans-Peter Schröcker. A new and efficient algorithm for the inverse kinematics of a general serial 6r manipulator. *Mechanism and Machine Theory*, 42(1):66–81, 2007. ISSN 0094-114X. doi: 10.1016/j.mechmachtheory.2006.02.001.

[20] Shuguang Qiao, Qizheng Liao, Shimin Wei, and Hai-Jun Su. Inverse kinematic analysis of the general 6r serial manipulators based on double quaternions. *Mechanism and Machine Theory*, 45(2):193–199, 2010. ISSN 0094-114X. doi: https://doi.org/10.1016/j.mechmachtheory.2009.05.013. URL https://www.sciencedirect.com/science/article/pii/S0094114X09001098.

[21] Fan Xiao, Gongfa Li, Du Jiang, Yuanmin Xie, Juntong Yun, Ying Liu, Li Huang, and Zifan Fang. An effective and unified method to derive the inverse kinematics formulas of general six-dof manipulator with simple geometry. *Mechanism and Machine Theory*, 159:104265, 2021. ISSN 0094-114X. doi: 10.1016/j.mechmachtheory.2021.104265.

[22] Fan Chen, Hong Ju, and Xinyu Liu. Inverse kinematic formula for a new class of 6r robotic arms with simple constraints. *Mechanism and Machine Theory*, 179:105118, 2023. ISSN 0094-114X. doi: 10.1016/j.mechmachtheory.2022.105118.

[23] Dianmu Zhang and Blake Hannaford. Ikbt: Solving symbolic inverse kinematics with behavior tree. *Journal of Artificial Intelligence Research*, 65:457–486, 2019.

[24] A. Malik, Y. Lischuk, T. Henderson, and R. Prazenica. A deep reinforcement-learning approach for inverse kinematics solution of a high degree of freedom robotic manipulator. *Robotics*, 11(2):44, 2022.

[25] J. Ames, J. Morgan, and G. Konidaris. Ikflow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022.

[26] OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2023. URL https://cdn.openai.com/papers/gpt-4.pdf.

[27] Hai-Jun Su. Analytical solvers for common algebraic equations arising in kinematics problems. *arXiv preprint arXiv:2509.01010*, 2025. URL https://arxiv.org/abs/2509.01010. Available at https://arxiv.org/abs/2509.01010.

# APPENDIX

## A HELPER FUNCTIONS FOR SOLVING TRIGONOMETRIC EQUATIONS

Here we detail the helper functions developed to solve common trigonometric equation types encountered in inverse kinematics derivations. Degenerate cases were carefully handled to avoid singularities and ensure all valid solutions are found. Extensive stress testing with random coefficients confirmed robustness and accuracy. These building blocks underpin the higher-level IK solvers presented in this paper.

### *A.1 Single Trigonometric Equation Solver*

The `solve_trig_eq()` function solves the trigonometric equation represents a linear combination of sine and cosine terms:

$$a\cos\theta + b\sin\theta + c = 0$$

**Solution Method:** The solver uses the Weierstrass substitution $t = \tan(\theta/2)$, which transforms the equation into a quadratic polynomial in $t$:

$$(c-a)t^2 + 2bt + (a+c) = 0$$

The two solutions for $\theta$ are then:

$$\theta_{1,2} = 2\arctan\left(\frac{-b \pm \sqrt{a^2+b^2-c^2}}{(c-a)}\right)$$

**Degenerate Cases:** The solver handles several degenerate cases. When $c - a = 0$, the equation reduces to a linear form $2bt + (a + c) = 0$, yielding a single solution $\theta = 2\arctan(-(a + c)/(2b))$ if $b \neq 0$. If $b = 0$ as well, the equation reduces to $a + c = 0$, leading to either infinite solutions (if $a + c = 0$) or no solution (if $a + c \neq 0$). The solver also checks for cases where $a^2 + b^2 - c^2 < 0$, indicating no real solutions.

**Expected Solutions**: Up to 2 real solutions in $[-\pi, \pi]$ (corresponding to the two branches of the arctangent function).

### *A.2 Single Angle Trigonometric System*

The `solve_trig_sys_single()` function solves systems of the form:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{Bmatrix} \cos\theta \\ \sin\theta \end{Bmatrix} = \begin{Bmatrix} C_1 \\ C_2 \end{Bmatrix}$$

**Solution Method:** When the coefficient matrix $A$ is invertible (i.e., $\det(A) \neq 0$), the system is solved directly by matrix inversion:

$$\begin{Bmatrix} \cos\theta \\ \sin\theta \end{Bmatrix} = \mathbf{A}^{-1} \begin{Bmatrix} C_1 \\ C_2 \end{Bmatrix}$$

then $\theta$ is computed using $\texttt{atan2}(\sin\theta, \cos\theta)$. The solution is validated by checking that $\cos^2\theta + \sin^2\theta = 1$ within numerical tolerance.

**Degenerate Cases:** Matrix $\mathbf{A}$ is singular (rank($\mathbf{A}$)=1), the solver selects the equation with the largest coefficient magnitude and uses `solve_trig_eq()` to obtain up to two solutions for $\theta$. If rank($\mathbf{A}$)=0, the system is either inconsistent (no solution) or indeterminate (infinite solutions), which the solver detects and handles accordingly.

**Expected Solutions**: one unique solution when $\mathbf{A}$ is invertible and the computed $(\cos\theta, \sin\theta)$ satisfies the unit circle constraint, or up to 2 solutions when $\mathbf{A}$ is singular (or zero solutions if the system is inconsistent).

### *A.3 Two-Angle Trigonometric System*

The `solve_trig_sys()` function solves systems of the form:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{Bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{Bmatrix} + \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \begin{Bmatrix} \cos\theta_2 \\ \sin\theta_2 \end{Bmatrix} = \begin{Bmatrix} C_1 \\ C_2 \end{Bmatrix}$$

with trigonometric equations in two angles $\theta_1$ and $\theta_2$.

**Solution Method:** When matrix $\mathbf{B}$ has full rank, use the following steps:

1. Express $\cos\theta_2$ and $\sin\theta_2$ linearly in terms of $\cos\theta_1$ and $\sin\theta_1$:
$$\begin{Bmatrix} \cos\theta_2 \\ \sin\theta_2 \end{Bmatrix} = \mathbf{B}^{-1}\left(\begin{Bmatrix} C_1 \\ C_2 \end{Bmatrix} - \mathbf{A}\begin{Bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{Bmatrix}\right)$$
2. Substitute into the unit circle constraint $\cos^2\theta_2 + \sin^2\theta_2 = 1$, yielding a nonlinear equation in $\cos\theta_1$ and $\sin\theta_1$
3. Use $t = \tan(\theta_1/2)$ to convert the equation into a quartic polynomial in $t$:
$$a_4t^4 + a_3t^3 + a_2t^2 + a_1t + a_0 = 0$$
4. Solve the quartic equation for $t$ and convert back to $\theta_1$.
5. Substitute each $\theta_1$ solution back into the linear system in Step 1, then solve for $\theta_2 = \texttt{atan2}(\sin\theta_2, \cos\theta_2)$

The quartic polynomial coefficients were derived symbolically using Mathematica and hardcoded in the Python implementation. Table 5 shows the coefficient expressions.

**Degenerate Cases:** When matrix $\mathbf{B}$ is singular, specialized methods are used:

- **Rank 0 (Zero Matrix)**: $\theta_2$ is free, solve $\mathbf{A}[\cos\theta_1, \sin\theta_1] = \mathbf{C}$ for $\theta_1$ using `solve_trig_sys_single()`. Expect up to 2 solutions for $\theta_1$ and $\theta_2$ is arbitrary.

Table 5: Quartic Polynomial Coefficients for Two-Angle Trigonometric System

| Coefficient | Expression |
|---|---|
| $a_4$ | $A_{11}^2B_{21}^2 + A_{11}^2B_{22}^2 - 2A_{11}A_{21}B_{11}B_{21} - 2A_{11}A_{21}B_{12}B_{22} - 2A_{11}B_{11}B_{21}C_1 - 2A_{11}B_{12}B_{22}C_1$ $+2A_{11}B_{21}^2C_1 + 2A_{11}B_{22}^2C_1 + A_{21}^2B_{11}^2 + A_{21}^2B_{12}^2 + 2A_{21}B_{11}^2C_2 - 2A_{21}B_{11}B_{21}C_1$ $+2A_{21}B_{12}^2C_2 - 2A_{21}B_{12}B_{22}C_1 - B_{11}^2B_{22}^2 + B_{11}^2C_2^2 + 2B_{11}B_{12}B_{21}B_{22} - 2B_{11}B_{21}C_1C_2$ $-B_{12}^2B_{21}^2 + B_{12}^2C_2^2 - 2B_{12}B_{22}C_1C_2 + B_{21}^2C_1^2 + B_{22}^2C_1^2$ |
| $a_3$ | $4(-A_{11}A_{12}B_{21}^2 - A_{11}A_{12}B_{22}^2 + A_{11}A_{22}B_{11}B_{21} + A_{11}A_{22}B_{12}B_{22} + A_{12}A_{21}B_{11}B_{21} + A_{12}A_{21}B_{12}B_{22}$ $+A_{12}B_{11}B_{21}C_2 + A_{12}B_{12}B_{22}C_2 - A_{12}B_{21}^2C_1 - A_{12}B_{22}^2C_1 - A_{21}A_{22}B_{11}^2 - A_{21}A_{22}B_{12}^2$ $-A_{22}B_{11}^2C_2 + A_{22}B_{11}B_{21}C_1 - A_{22}B_{12}^2C_2 + A_{22}B_{12}B_{22}C_1)$ |
| $a_2$ | $2(-A_{11}^2B_{21}^2 - A_{11}^2B_{22}^2 + 2A_{11}A_{21}B_{11}B_{21} + 2A_{11}A_{21}B_{12}B_{22} + 2A_{12}^2B_{21}^2 + 2A_{12}^2B_{22}^2$ $-4A_{12}A_{22}B_{11}B_{21} - 4A_{12}A_{22}B_{12}B_{22} - A_{21}^2B_{11}^2 - A_{21}^2B_{12}^2 + 2A_{22}^2B_{11}^2 + 2A_{22}^2B_{12}^2$ $-B_{11}^2B_{22}^2 + B_{11}^2C_2^2 + 2B_{11}B_{12}B_{21}B_{22} - 2B_{11}B_{21}C_1C_2 - B_{12}^2B_{21}^2 + B_{12}^2C_2^2$ $-2B_{12}B_{22}C_1C_2 + B_{21}^2C_1^2 + B_{22}^2C_1^2)$ |
| $a_1$ | $4(A_{11}A_{12}B_{21}^2 + A_{11}A_{12}B_{22}^2 - A_{11}A_{22}B_{11}B_{21} - A_{11}A_{22}B_{12}B_{22} - A_{12}A_{21}B_{11}B_{21} - A_{12}A_{21}B_{12}B_{22}$ $+A_{12}B_{11}B_{21}C_2 + A_{12}B_{12}B_{22}C_2 - A_{12}B_{21}^2C_1 - A_{12}B_{22}^2C_1 + A_{21}A_{22}B_{11}^2 + A_{21}A_{22}B_{12}^2$ $-A_{22}B_{11}^2C_2 + A_{22}B_{11}B_{21}C_1 - A_{22}B_{12}^2C_2 + A_{22}B_{12}B_{22}C_1)$ |
| $a_0$ | $A_{11}^2B_{21}^2 + A_{11}^2B_{22}^2 - 2A_{11}A_{21}B_{11}B_{21} - 2A_{11}A_{21}B_{12}B_{22} + 2A_{11}B_{11}B_{21}C_2 + 2A_{11}B_{12}B_{22}C_2$ $-2A_{11}B_{21}^2C_1 - 2A_{11}B_{22}^2C_1 + A_{21}^2B_{11}^2 + A_{21}^2B_{12}^2 - 2A_{21}B_{11}^2C_2 + 2A_{21}B_{11}B_{21}C_1$ $-2A_{21}B_{12}^2C_2 + 2A_{21}B_{12}B_{22}C_1 - B_{11}^2B_{22}^2 + B_{11}^2C_2^2 + 2B_{11}B_{12}B_{21}B_{22} - 2B_{11}B_{21}C_1C_2$ $-B_{12}^2B_{21}^2 + B_{12}^2C_2^2 - 2B_{12}B_{22}C_1C_2 + B_{21}^2C_1^2 + B_{22}^2C_1^2$ |

- **Rank 1**: In this case, the two angles can be decoupled. Use row reduction to eliminate variables, then solve sequentially for $\theta_1$ and $\theta_2$. Solve the one row reduced equation to solve for $\theta_1$ by the single-angle trigonometric equations using `solve_trig_eq()`. We obtain two solutions of $\theta_1$ and then substitute each solution of $\theta_1$ into the other equation solve for $\theta_2$. We expect up to 4 combinations of solutions.

**Expected Solutions**: Up to 4 solutions, depending on system degeneracy and solution branches.

### *A.4 Bilinear Two-Angle System*

The `solve_bilinear_sys()` function solves bilinear systems of the form:

$$\begin{bmatrix} a_{10} & a_{11c} & a_{11s} & a_{12c} & a_{12s} & a_{1cc} & a_{1cs} & a_{1sc} & a_{1ss} \\ a_{20} & a_{21c} & a_{21s} & a_{22c} & a_{22s} & a_{2cc} & a_{2cs} & a_{2sc} & a_{2ss} \end{bmatrix} \mathbf{m} = \mathbf{0}$$

where the monomial vector $\mathbf{m}$:

$$\mathbf{m} = (1, c_1, s_1, c_2, s_2, c_1c_2, c_1s_2, s_1c_2, s_1s_2)^T$$

where $c_1 = \cos\theta_1$, $s_1 = \sin\theta_1$, $c_2 = \cos\theta_2$, $s_2 = \sin\theta_2$. This is a general case of the two-angle trigonometric system illustrated in the previous section.

**Solution Method:**

1. Transform the trigonometric equations into bivariate polynomials $p_1$ and $p_2$, both of degree 2 in $t_1 = \tan(\theta_1/2)$ and $t_2 = \tan(\theta_2/2)$

$$p_k(t_1, t_2) = \sum_{i=0}^{2}\sum_{j=0}^{2} p_{k,ij}\, t_1^i\, t_2^j, \quad k = 1, 2$$

   The coefficients $p_{k,ij}$ are computed from the bilinear coefficients $a_{k*}$ as shown in Table 6.

2. Construct the Sylvester resultant matrix from $p_1$ and $p_2$ to eliminate $t_2$, resulting in a matrix polynomial

$$|\mathbf{M}(t_1)| = |\mathbf{M}_0 + \mathbf{M}_1 t_1 + \mathbf{M}_2 t_1^2|$$

   where $\mathbf{M}_0$, $\mathbf{M}_1$, and $\mathbf{M}_2$ are $4 \times 4$ matrices formed from the coefficients of $p_1$ and $p_2$.

3. Solve the generalized eigenvalue problem for the matrix polynomial $|\mathbf{M}(t_1)| = 0$. When $\mathbf{M}_2$ is well-conditioned,construct the companion matrix as

$$C = \begin{bmatrix} -\mathbf{M}_2^{-1}\mathbf{M}_1 & -\mathbf{M}_2^{-1}\mathbf{M}_0 \\ I & 0 \end{bmatrix}$$

   where $I$ is the $4 \times 4$ identity matrix. The eigenvalues of $C$ are the roots $t_1$ of the matrix polynomial $|\mathbf{M}(t_1)| = 0$. Otherwise, construct generalized eigenvalue matrices $A = \begin{bmatrix} -\mathbf{M}_1 & -\mathbf{M}_0 \\ I & 0 \end{bmatrix}$ and $B = \begin{bmatrix} \mathbf{M}_2 & 0 \\ 0 & I \end{bmatrix}$, then find eigenvalues $\lambda$ such that

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{B}\mathbf{v}$$

   The real eigenvalues $\lambda$ are the roots $t_1$, avoiding direct computation of and finding roots of an 8th-degree polynomial. Note that the infinite or numerically large eigenvalues are valid roots corresponding to $\theta_1 = 180°$.

4. For each $t_1$ solution, convert back to $(c_1, s_1)$ and substitute them back into the original two equations, solve for $\theta_2$ using the robust `solve_trig_sys_single()` function.

The coefficients of the polynomials $p_k(t_1, t_2)$ (shown in Table 6) were derived symbolically using Mathematica and hardcoded in numerical implementation.

Table 6: Coefficients for polynomial $p_k(t_1, t_2)$ $(k = 1, 2)$

| $i \backslash j$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | $a_{k0} + a_{k1c} + a_{k2c} + a_{kcc}$ | $2(a_{k2s} + a_{kcs})$ | $a_{k0} + a_{k1c} - a_{k2c} - a_{kcc}$ |
| 1 | $2(a_{k1s} + a_{ksc})$ | $4a_{kss}$ | $2(a_{k1s} - a_{ksc})$ |
| 2 | $a_{k0} - a_{k1c} + a_{k2c} - a_{kcc}$ | $2(a_{k2s} - a_{kcs})$ | $a_{k0} - a_{k1c} - a_{k2c} + a_{kcc}$ |

**Expected Solutions**: Up to 8 solutions for general coefficients, though typically 4 real solutions for 6R robots.

## B Coefficient Expressions of IK of 6R Robots

Table 7: Coefficient Expressions of Eq. (4)

| |
|---|
| **A** matrix elements: |
| $A_{11} = -p_y s_{\alpha 1}$ |
| $A_{12} = p_x s_{\alpha 1}$ |
| $A_{21} = -2a_1 p_x + 2d_2 p_y s_{\alpha 1}$ |
| $A_{22} = -2a_1 p_y - 2d_2 p_x s_{\alpha 1}$ |
| $B_{11} = d_4 s_{\alpha 2} s_{\alpha 3}$ |
| $B_{12} = -a_3 s_{\alpha 2}$ |
| $B_{21} = -2a_2 a_3$ |
| $B_{22} = -2a_2 d_4 s_{\alpha 3}$ |
| $C_1 = d_2 - p_z c_{\alpha 1} + c_{\alpha 2}(d_3 + d_4 c_{\alpha 3})$ |
| $C_2 = -(a_1^2 - a_2^2 - a_3^2 + d_2^2 - d_3^2 - d_4^2 + p_x^2 + p_y^2 + p_z^2$ $-2d_2 p_z c_{\alpha 1} - 2d_3 d_4 c_{\alpha 3})$ |

Table 8: Bilinear Coefficient Expressions of Eq. (14)

| Coefficient Expression |
|---|
| $\mathbf{z}_4 \cdot \mathbf{z}_5 - c_{\alpha 4} = 0:$ |
| $a_{10} = -c_{\alpha 4} + r_{33} c_{\alpha 1} c_{\alpha 5}$ |
| $a_{11c} = -r_{23} c_{\alpha 5} s_{\alpha 1}$ |
| $a_{11s} = r_{13} c_{\alpha 5} s_{\alpha 1}$ |
| $a_{12c} = r_{32} c_{\alpha 1} s_{\alpha 5}$ |
| $a_{12s} = r_{31} c_{\alpha 1} s_{\alpha 5}$ |
| $a_{1cc} = -r_{22} s_{\alpha 1} s_{\alpha 5}$ |
| $a_{1cs} = -r_{21} s_{\alpha 1} s_{\alpha 5}$ |
| $a_{1sc} = r_{12} s_{\alpha 1} s_{\alpha 5}$ |
| $a_{1ss} = r_{11} s_{\alpha 1} s_{\alpha 5}$ |
| $(\mathbf{P}_{05}^R - \mathbf{P}_{05}^L) \cdot \mathbf{z}_2 = 0:$ |
| $a_{20} = d_2 + d_3 + d_4 + d_5 c_{\alpha 4} - p_z c_{\alpha 1}$ |
| $a_{21c} = p_y s_{\alpha 1}$ |
| $a_{21s} = -p_x s_{\alpha 1}$ |
| $a_{22c} = a_5 r_{31} c_{\alpha 1}$ |
| $a_{22s} = -a_5 r_{32} c_{\alpha 1}$ |
| $a_{2cc} = -a_5 r_{21} s_{\alpha 1}$ |
| $a_{2cs} = a_5 r_{22} s_{\alpha 1}$ |
| $a_{2sc} = a_5 r_{11} s_{\alpha 1}$ |
| $a_{2ss} = -a_5 r_{12} s_{\alpha 1}$ |